Menu
My projects
Holy Graph
List projects
Available Cursus
Your projects
camagru
libasm
malloc
zen
woody-woodpacker

SCALE FOR PROJECT H42N42

You should evaluate 1 student in this team

Git repository

`git@vogsphere-v2.42b`

Introduction

For the smooth running of this evaluation, please respect the following rules:

- Remain polite, kind, respectful and constructive whatever happens during this conversation. It's a matter of confidence between you and the 42 community.

- Highlight the potential problems you've had with the work you're presented to the person or the group you're grading, and take the time to talk about and discuss those issues.

- Accept the fact that the project subject or required functions might lead to different interpretations. Listen to your discussion partner's perspective with an open mind (are they right or wrong?) and grade them as fairly as possible.
42's teaching methods only make sense if peer-evaluation is taken seriously.

Disclaimer

Ocsigen is open source software developed by CNRS, University of Paris Diderot, and Inria public research labs. It is developed in collaboration with BeSport SAS. It is a collaborative project that grows with each user's contributions.

This project will only give you a small glimpse of what Ocsigen can do and it won't use its most significant functionalities (client-server programming, mobile applications...). These points will be tackled in a coming project.

This project is an introduction to web programming in OCaml on the client side with Ocsigen. It helps us evaluate:

- First, your ability to approach a new technology with an open mind, to find your way in a substantial documentation in English and to join a community around an open software.

- Your understanding of web programming basics with HTML, CSS, and DOM on the client side.

- Your understanding of programming basics in OCaml, TyXML, and List libraries, and interaction between OCaml and the DOM.

- Your ability to create a clean and user-friendly application with playable rules.

Guidelines

- You must only evaluate what you will find in the student's or group's GiT repository.

- Take the time to check that the GiT repository matches the student or group and the project.

- Double check that no malicious alias was used to mislead you and make you grade something different from the official repository content.

- If a script intended to help evaluate the project is supplied by either side, the other side must strictly check it to avoid nasty surprises.

- If the evaluating student has not yet taken this project, they must read the project subject in its entirety before starting the evaluation.

- Use the flags available on this grading system to signal an empty or non-functional project, a norm violation, cheating, etc. In that case, evaluation stops and the final grade is 0 (or -42 if it's a cheating problem). However, if it's not a cheating problem, you are encouraged to continue discussing the work that has been done (or not done) in order to identify the issues that led to this situation and avoid them next time.

Attachments

subject.pdf

Docker Setup & Deployment

This section evaluates the Docker configuration and deployment setup. The project must be easily deployable using Docker.

Docker Configuration

Verify that the project includes both a Dockerfile and docker-compose.yml file. (2 points) Check that the Dockerfile uses an appropriate OCaml base image and installs all necessary dependencies (Ocsigen, Eliom, ?s_ol_ocaml). (3 points)

Rate it from 0 (failed) through 5 (excellent)

5

Application Launch

Run `docker-compose up --build` and verify that the application builds successfully without errors. The application should be accessible in a browser at the specified port (default: http://localhost:8080). The entire setup should work without any manual intervention.

Yes | No

Playability and adherence to guidelines

This part will evaluate the mandatory part of the subject and check that all the points required by the subject are valid.

Creature movements

Creature movements must be smooth and realistic (especially rebounds). There should be no creatures outside the play area, even berserk ones when they're close to the edge. Creatures must be able to reach the edges of the play area.

Yes | No

Contamination

Berserk creatures must die when they reach 4x their initial size (approximately 2 minutes 30 seconds after becoming berserk), and mean creatures must die after 1 minute. (1 point) Touch contamination with the river or a sick creature must be implemented. (2 points) Distance calculation must be correct (no contamination without touch, potential contamination if touch). (2 points)

Rate it from 0 (failed) through 5 (excellent)

5

Berserk and mean creatures

Some sick creatures will go berserk (10% chance checked every 10 seconds). They grow 10% bigger every 10 seconds, thus increasing the risk of touching another creature, and die when reaching 4x their initial size. (2 points) Berserk size must be taken into account in the contamination's calculation. (1 point) Some sick creatures must get mean (10% chance checked every 10 seconds). They run after the healthy creatures to contaminate them and die after 1 minute. Berserk and mean creatures cannot be grabbed or taken to the hospital. (2 points)

Rate it from 0 (failed) through 5 (excellent)

5

Hospital

When sick creatures are taken to the hospital, they must be healed when they get out.

Yes | No

Game

The difficulty must evolve and the game must be playable.(2 points) No new creature should spawn if there is no healthy creature left.(3 points)

Rate it from 0 (failed) through 5 (excellent)

5

Technical points

Competition

Each creature must be animated by an LWT thread, excluding any other kind of concurrency (don't use a Javascript function like setTimeout in particular). Creature movement must be smooth (not jerky).

Yes | No

Handling HTML

Each creature must be an HTML element. All dynamically generated elements of the page must be created using the TyXML library (such as Eliom_content.Html.D).

Yes | No

Interacting with the DOM

When there are no more creatures on the map, a message must appear indicating the game is over.

Yes | No

Mouse events

Mouse events must be exclusively programmed using the Lwt_js_events library. Creatures should be easily movable and droppable anywhere on the page. Test dropping creatures outside the game area and outside the window.

Yes | No

Code Quality & Documentation

This section evaluates the overall code quality, project structure, and documentation.

Project Structure

Verify that the project follows a clear and logical structure with separate directories for source code (src/), static assets (static/), and configuration files at the root. The Makefile should be present with all required rules (all, clean, fclean, re) and should not recompile unnecessarily.

Yes | No

Documentation

Check that a comprehensive README.md file exists with clear setup instructions, prerequisites, how to run the application using Docker, game rules explanation, and any additional features implemented. (3 points) The code should include helpful comments explaining complex logic. (2 points)

Rate it from 0 (failed) through 5 (excellent)

Bonus

Bonuses will only be taken into account if the mandatory part is PERFECT. Perfect means it showed no errors or random behavior during testing and evaluation. Each bonus is worth 5 points. Students can implement up to 5 bonuses for a maximum of 25 bonus points.

Bonus 1 - Advanced Graphics & Visual Effects

Verify that the game has enhanced visual appeal with professional graphics:

- Custom sprites or images for different Creet types (1 point)
- Smooth CSS animations or transitions for state changes (1 point)
- Visual effects such as particle effects, glow/pulse effects, shadows, or animated background (2 points)
- Cohesive visual theme with consistent styling (1 point)

All elements should work together to create a polished, professional appearance. The bonus is awarded based on the quality and completeness of the implementation.

Rate it from 0 (failed) through 5 (excellent)

Bonus 2 - Interactive Control Panel

Check that a comprehensive control panel is implemented:

- Sliders to adjust game parameters (speed, contamination probability, reproduction rate, etc.) (2 points)
- Buttons for pause/resume, reset, and game mode toggles (1 point)
- Real-time information display (creature counts, game duration, creatures saved) (1 point)

- Well-designed, intuitive interface that doesn't obstruct gameplay (1 point)

All parameter changes must take effect immediately without requiring a restart. Test multiple parameters to ensure they work correctly.

Rate it from 0 (failed) through 5 (excellent)

Bonus 3 - Sound Effects & Audio

Verify that audio features are implemented:

- Background music that loops continuously (1 point)
- Sound effects for key events (contamination, healing, death, game over, reproduction) (2 points)
- Audio controls (mute/unmute, volume adjustment for music and effects separately) (1 point)
- Audio files load properly without performance issues (1 point)

The audio should enhance the experience without being annoying. Test multiple sound effects and verify volume controls work correctly.

Rate it from 0 (failed) through 5 (excellent)

Bonus 4 - Statistics & Scoring System

Check that a statistics and scoring system is implemented:

- Track and display statistics (survival time, creatures saved/lost, max creatures alive, berserk/mean encounters) (2 points)
- Calculate a score based on survival duration, creatures saved, and efficiency (1 point)
- Statistics dashboard that can be toggled on/off (1 point)
- Detailed game over screen with final statistics and score (1 point)

Optionally, check for a local leaderboard using localStorage. Statistics should be presented in a clear, visually appealing format.

Rate it from 0 (failed) through 5 (excellent)

Bonus 5 - Optimized Collision Detection

Verify that collision detection has been optimized using spatial partitioning:

- Implementation of quadtree, grid-based spatial hashing, or similar algorithm (2 points)
- Significant performance improvement demonstrated (game handles 100+ creatures smoothly) (2 points)
- Ability to toggle between optimized and naive detection, or FPS counter showing performance (1 point)

Ask the student to explain their algorithm. The implementation must be correct and not introduce collision detection bugs. Code should be well-documented.

Rate it from 0 (failed) through 5 (excellent)

Ratings

Don’t forget to check the flag corresponding to the defense

✔ Ok | ★ Outstanding project

Empty work | No author file | Invalid compilation | Norme

Cheat | Crash | Incomplete group | Forbidden function

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation